MANIPAL UNIVERSITY JAIPUR

School of Computer Science & Engineering
Department of Computer Science & Engineering

A Research Project Report on

Store-Level Retail Demand Forecasting

Submitted in partial fulfilment of the requirements of the course
Predictive Analysis

Project Title	Store-Level Retail Demand Forecasting
Department	Computer Science & Engineering
Name of Student	Prasham Jain
Registration Number	2427030155
Section	J (Semester 5)
Date of Start	
Date of End	
Academic Session	2026–2027

Manipal University Jaipur
Jaipur, Rajasthan, India
2026

Certificate of Approval

This is to certify that the research project report entitled **"Store-Level Retail Demand Forecasting"**, submitted by **Prasham Jain** (Registration No. **2427030155**, Section J, Semester 5) to the Department of Computer Science & Engineering, Manipal University Jaipur, in partial fulfilment of the requirements of the course **Predictive Analysis**, is a record of original work carried out by the student under my supervision during the academic session 2026–2027.

The work embodied in this report has been examined and is approved for submission. To the best of my knowledge, it has not been submitted elsewhere for the award of any other credit, degree or diploma.

Project Supervisor	**Head of Department**
Name:	Name:
Department of Computer Science & Engineering	Department of Computer Science & Engineering
Manipal University Jaipur	Manipal University Jaipur

Date: ..... Place: Jaipur

Declaration by the Student

I, **Prasham Jain**, Registration No. **2427030155**, a student of Section J, Semester 5, Department of Computer Science & Engineering, Manipal University Jaipur, hereby declare that the research project report entitled **"Store-Level Retail Demand Forecasting"** is a record of my own work, carried out as part of the course **Predictive Analysis** during the academic session 2026–2027.

I further declare that:

- the work presented in this report is original and has not been submitted, in part or in full, for any other course, degree or diploma;
- all datasets, software libraries and published works used in this project have been acknowledged and cited in the References;
- every numerical result reported here was produced by running the project code against the real datasets, and none has been estimated, interpolated or altered.

_____

Prasham Jain
Registration No. 2427030155
Section J, Semester 5

Date: ..... Place: Jaipur

Acknowledgement

I would like to express my sincere gratitude to my project supervisor and to the faculty of the Department of Computer Science & Engineering, Manipal University Jaipur, for their guidance throughout the **Predictive Analysis** course. Their emphasis on sound validation and honest evaluation shaped the way this project was designed.

I thank the Head of the Department and the School of Computer Science & Engineering for providing the academic environment and resources that made this work possible.

I am grateful to Rossmann and to Kaggle for making the Rossmann Store Sales data publicly available for research and learning, and to the contributors of the Walmart Sales dataset used in the multi-retailer experiment. I also acknowledge the developers of the open-source tools on which this project is built, including Python, pandas, scikit-learn, LightGBM, SHAP, statsmodels and FastAPI.

Finally, I thank my family and friends for their constant encouragement and support.

Prasham Jain
2427030155

Abstract

Retailers commonly plan inventory, staffing and procurement using manual estimates or simple historical averages, which fail precisely when accuracy matters most: during promotions, public holidays and seasonal peaks. This project designs, implements and evaluates a machine-learning system that forecasts daily sales for individual stores, using the Rossmann Store Sales dataset (1,017,209 daily records from 1,115 stores between January 2013 and July 2015).

The pipeline cleans and merges the raw data without dropping any rows, decomposes sales into trend and weekly seasonality, and builds 37 input features, including lag and rolling-window statistics that are constructed so that a feature can only see a store's strictly earlier history. An executable leakage check independently recomputes these features from raw history and confirms they match. Four models are compared: two naive baselines (seasonal-naive and a 7-day moving average), Ridge regression, and a single global LightGBM gradient-boosting model trained on log-transformed sales. All models are evaluated with five-fold expanding-window walk-forward validation, where each fold scores a 42-day block of dates the model never trained on.

LightGBM achieved the lowest error in all five folds, with a mean weighted absolute percentage error (WMAPE) of 7.87% and a mean absolute scaled error (MASE) of 0.240, compared with 11.87% for Ridge regression and 31.19% for the seasonal-naive baseline. SHAP analysis shows that store identity, calendar position, recent sales momentum and promotions drive the predictions. The trained model produces a 48-day recursive forecast for the 856 stores in the real Kaggle test window, with no missing or negative predictions. Two extensions go beyond the original synopsis: an interactive FastAPI web application that re-runs forecasts, explanations, leakage checks and validation on demand, and a multi-retailer experiment in which adding Walmart's sales history reduced Rossmann's weekly WMAPE from 7.11% to 7.01% and MASE from 0.255 to 0.190.

Keywords: demand forecasting, retail analytics, time series, gradient boosting, LightGBM, walk-forward validation, data leakage, SHAP, explainable AI.

Table of Contents

Certificate of Approval ii
Declaration by the Student iii
Acknowledgement iv
Abstract v
List of Figures viii
List of Tables ix
List of Abbreviations x
Chapter 1: Introduction 1
1.1 Background of the Study 1
1.2 Problem Statement 1
1.3 Research Motivation and Significance 2
1.4 Research Aim and Objectives 2
1.5 Research Questions / Hypotheses 3
1.6 Scope of the Study 3
1.7 Organization of the Report 3
Chapter 2: Literature Review 5
2.1 Introduction 5
2.2 Review of Existing Studies 5
2.3 Comparison of Existing Methods 6
2.4 Identified Research Gaps 7
2.5 Chapter Summary 7
Chapter 3: Research Methodology 9
3.1 Research Design 9
3.2 Dataset / Data Sources 9
3.3 Data Collection and Preprocessing 10
3.4 Proposed Methods / Algorithms 11
3.5 Tools, Software, and Experimental Setup 14
3.6 Evaluation Metrics 14
3.7 Ethical Considerations 15
Chapter 4: System Design and Implementation 16
4.1 System Architecture / Proposed Framework 16
4.2 Workflow or Process Diagram 16
4.3 Model / System Design 17
4.4 Implementation Details 19

4.5 Computational Requirements 20
Chapter 5: Results and Discussion 21
5.1 Experimental Results 21
5.2 Quantitative and Qualitative Evaluation 24
5.3 Comparison with Baseline / Existing Methods 25
5.4 Interpretation of Findings 26
5.5 Limitations and Threats to Validity 29
Chapter 6: Conclusion and Future Work 31
6.1 Summary of the Research 31
6.2 Major Findings 31
6.3 Research Contributions 31
6.4 Limitations 32
6.5 Future Research Directions 32
References 33
Appendices 35
Appendix A: Leakage-safe feature construction 35
Appendix B: Walk-forward validation splits 35
Appendix C: LightGBM configuration 36
Appendix D: Full validation results by fold 36
Appendix E: Additional SHAP dependence plots 37
Code and Dataset Availability 38

List of Figures

Figure 1 System architecture 16
Figure 2 End-to-end project workflow 17
Figure 3 STL decomposition of total daily sales into trend, weekly seasonal and residual components 21
Figure 4 Mean sales by day of the week 22
Figure 5 Mean sales with and without a promotion 22
Figure 6 Sales distribution by store type 22
Figure 7 WMAPE by model and validation fold; labels show the LightGBM value 23
Figure 8 Total chain demand: last eight weeks of actual sales and the 48-day forecast . . . 24
Figure 9 LightGBM predictions against actual sales for three sample stores 25
Figure 10 Rossmann-only versus combined (Rossmann + Walmart) training, by fold 26
Figure 11 LightGBM feature importance (gain) 27
Figure 12 SHAP summary of feature effects on individual predictions 28
Figure 13 SHAP dependence: Promo 28
Figure 14 SHAP dependence: DayOfWeek 28
Figure 15 SHAP dependence: CompetitionDistance 37
Figure 16 SHAP dependence: StoreType 37
Figure 17 SHAP dependence: state holiday 37
Figure 18 SHAP dependence: 7-day rolling mean 37

List of Tables

Table 1 Comparison of forecasting method families 7
Table 2 Project stages and corresponding notebooks 9
Table 3 Summary of the datasets used 10
Table 4 Model input features 12
Table 5 Training and validation sizes for each walk-forward fold 13
Table 6 Software tools and libraries 14
Table 7 Repository structure 19
Table 8 Web application panels and API routes 19
Table 9 Computational requirements 20
Table 10 Mean validation error across five walk-forward folds (lower is better) 22
Table 11 WMAPE by fold for each model 23
Table 12 Multi-retailer experiment: Rossmann weekly validation error 26
Table 13 Complete walk-forward validation results 36

List of Abbreviations

Abbreviation	Full form
API	Application Programming Interface
ARIMA	Autoregressive Integrated Moving Average
CPI	Consumer Price Index
CSV	Comma-Separated Values
CV	Cross-Validation
EDA	Exploratory Data Analysis
GBDT	Gradient-Boosted Decision Trees
LightGBM	Light Gradient Boosting Machine
MAE	Mean Absolute Error
MASE	Mean Absolute Scaled Error
ML	Machine Learning
RMSE	Root Mean Squared Error
SHAP	SHapley Additive exPlanations
STL	Seasonal-Trend decomposition using Loess
WMAPE	Weighted Mean Absolute Percentage Error

Chapter 1: Introduction

1.1 Background of the Study

Demand forecasting sits at the centre of retail operations. Every decision about how much stock to order, how many staff to roster and when to schedule deliveries depends on an estimate of how much each store will sell in the coming days and weeks. Errors in either direction are expensive: under-forecasting empties shelves and loses sales, while over-forecasting ties up working capital in surplus inventory and, for perishable goods, leads directly to waste [16].

Traditionally, these estimates are produced by store managers or planners using experience, last year's figures or simple moving averages. Such methods work reasonably well in ordinary weeks but break down around the events that most affect demand, such as promotions, public and school holidays, competitor openings and the weekly shopping cycle. Modern retailers record every transaction, so the historical data needed to learn these patterns already exists. The challenge is converting that history into accurate, store-specific forecasts at scale.

Machine-learning methods, and gradient-boosted decision trees in particular, have become the dominant approach for this kind of problem. In large public forecasting competitions such as the Kaggle Rossmann competition and the M5 competition, tree-based ensembles trained on lag and calendar features consistently outperformed classical statistical models [8], [9]. This project applies these methods to the Rossmann Store Sales dataset [19], a real record of daily sales from 1,115 drug stores operated by Rossmann in Germany.

1.2 Problem Statement

Retail organisations frequently rely on manual estimation or simple historical averages to plan inventory, staffing and procurement, and these estimates tend to be inaccurate during promotional periods, public holidays and seasonal peaks. Under-forecasting leads to stock-outs and lost revenue, while over-forecasting results in excess inventory, higher holding costs and wastage of perishable goods. Manually analysing sales history, promotional calendars and seasonal patterns across more than a thousand stores is time-consuming and inconsistent, and does not scale.

There is therefore a need for a predictive-analysis solution that can process historical sales data, learn trend, seasonal and promotion-driven patterns, and accurately forecast future demand for each individual store in advance. Such a solution should support proactive inventory planning and informed operational decisions, and provide a reliable, model-driven alternative to manual estimation.

1.3 Research Motivation and Significance

The motivation for this work is both practical and methodological.

Practical significance. Even a few percentage points of forecast error, multiplied across 1,115 stores and every day of the year, translate into a large volume of lost sales or wasted stock. A model that reliably beats the naive methods used in practice gives planners a measurable improvement, not merely a more sophisticated one.

Methodological significance. Time-series forecasting is unusually vulnerable to evaluation mistakes. Randomly shuffled cross-validation, features computed with access to future values, and variables that are only known after the fact (such as the number of customers on the day being forecast) can all make a model look far more accurate than it will be in production [13], [14], [18]. A large part of this project's contribution is demonstrating a pipeline in which such leakage is prevented by construction and then verified by executable checks.

Educational significance. The project exercises the complete predictive-analysis workflow taught in the course: data acquisition, cleaning, exploratory analysis, feature engineering, model training, validation, interpretation and deployment, applied to a real retail problem.

1.4 Research Aim and Objectives

The aim of this project is to design, implement and rigorously evaluate a machine-learning system that forecasts daily store-level retail demand more accurately than standard baseline methods.

The specific objectives, taken from the approved project synopsis, are:

1. To analyse the retail sales dataset and identify relevant temporal, promotional and store-level features.
2. To perform data cleaning, preprocessing and exploratory data analysis, including decomposition of trend and seasonal components.
3. To select and engineer features such as lag variables, rolling-window statistics and calendar indicators that significantly influence demand.
4. To train suitable machine-learning models, such as regression and gradient-boosting algorithms, for demand forecasting.
5. To evaluate and compare model performance against baseline forecasting methods using RMSE, MAE, MASE and WMAPE under walk-forward validation.
6. To interpret model results and identify the key factors contributing to variation in demand.
7. To demonstrate a complete predictive-analysis pipeline from raw data to actionable forecast.

Two further objectives were added during the project: (8) to make the pipeline interactive through a live web application, and (9) to test whether training data from a second retailer improves forecasts for the first.

1.5 Research Questions / Hypotheses

The project is organised around four research questions:

- **RQ1.** Does a gradient-boosting model with engineered lag, rolling and calendar features forecast daily store sales more accurately than naive seasonal baselines and a linear regression model?
- **RQ2.** Which factors (store attributes, calendar effects, promotions, competition or recent sales history) contribute most to the model's predictions?
- **RQ3.** Can the trained model produce a complete, valid multi-week forecast on genuinely unseen future dates?
- **RQ4.** Does adding the sales history of a different retailer, with no overlapping stores or dates, improve forecasts for the original retailer?

The corresponding working hypothesis for RQ1 was set as an explicit acceptance criterion before modelling began: **the final model must beat every baseline on WMAPE in at least four of five walk-forward folds and achieve a mean MASE below 1.0** (that is, it must outperform the seasonal-naive forecast on the scaled measure).

1.6 Scope of the Study

The study covers daily, store-level sales forecasting for the 1,115 Rossmann stores using only the information supplied with the dataset: past sales, store attributes, promotions, holidays and competition. The forecast target is total daily sales per store, not individual product lines. External data such as weather, local events or macroeconomic indicators are outside the scope of the primary model. The multi-retailer experiment (Chapter 5) operates at weekly grain, because the Walmart data is only available weekly, and is reported as a separate experiment. The web application is a local demonstration, not a production deployment.

1.7 Organization of the Report

The remainder of this report is organised as follows. Chapter 2 reviews existing forecasting methods and identifies the gaps this project addresses. Chapter 3 describes the research design, datasets, preprocessing, algorithms, tools and evaluation metrics. Chapter 4 presents the system architecture, workflow and implementation, including the web application. Chapter 5 reports and discusses the experimental results, the forecasting demonstration and the multi-retailer experiment, together with limitations and threats to validity. Chapter 6 concludes the report and

outlines directions for future work. References, appendices and code and dataset availability follow.

Chapter 2: Literature Review

2.1 Introduction

Sales forecasting has been studied for decades, and the literature spans classical statistical models, machine-learning methods and, more recently, deep neural networks. This chapter reviews the main families of methods relevant to store-level retail forecasting, compares their strengths and weaknesses, and summarises the evaluation practices recommended in the forecasting literature. It closes by identifying the gaps that motivate the design of this project.

2.2 Review of Existing Studies

2.2.1 Classical statistical methods

The foundations of time-series forecasting are the exponential smoothing family and the Box–Jenkins ARIMA methodology [1], [2]. These methods model a single series at a time using its own past values, trend and seasonality, and remain strong benchmarks for smooth, regular series. Decomposition methods such as STL (Seasonal-Trend decomposition using Loess) [3] separate a series into trend, seasonal and residual components, which is valuable for understanding the structure of demand. Prophet [4] extended this decomposition idea into a practical forecasting tool with explicit holiday effects. The main limitation of these approaches in retail is that each store must be modelled separately, and incorporating many external drivers (promotions, competition, store type) is awkward.

2.2.2 Machine-learning and gradient-boosting methods

Gradient boosting [7] builds an ensemble of decision trees, each correcting the errors of the previous ones. Efficient implementations such as XGBoost [6] and LightGBM [5] can be trained on millions of rows, handle non-linear interactions automatically and accept categorical variables directly. When a forecasting problem is reframed as supervised regression, with lagged sales, rolling statistics and calendar indicators as inputs, these models can learn patterns shared across many stores at once.

Empirical evidence for this approach is strong. Bojer and Meldgaard's review of Kaggle forecasting competitions, including Rossmann, found that gradient-boosted trees with careful feature engineering were the most consistently successful approach [9]. In the M5 competition on Walmart product sales, the top-performing solutions were dominated by LightGBM models [8]. Montero-Manso and Hyndman showed theoretically and empirically that a single **global** model trained across many related series can match or outperform one **local** model per series, while using far fewer parameters per series [22].

2.2.3 Deep-learning methods

Neural approaches include DeepAR [11], an autoregressive recurrent network that learns a global probabilistic model across many related series, and entity embeddings [10], which learn dense vector representations of categorical variables. The entity-embedding approach was introduced through the Rossmann competition itself, where it placed third. These methods can be powerful, but they typically require more tuning and computation than gradient boosting, and their advantage on tabular retail data with strong engineered features is not consistent [8], [9].

2.2.4 Evaluation methodology

The forecasting literature places strong emphasis on correct evaluation. Tashman [13] reviewed out-of-sample testing and recommended rolling-origin (walk-forward) evaluation over a single fixed hold-out. Bergmeir and Benítez [14] showed that standard shuffled cross-validation is generally inappropriate for time series. Kaufman et al. [18] formalised **data leakage**, where information unavailable at prediction time enters training, as a major cause of over-optimistic results. On error measures, Hyndman and Koehler [12] showed that percentage errors such as MAPE are undefined or unstable when actual values are zero, a common situation on retail closing days, and proposed MASE, which scales errors by those of a naive forecast. Fildes et al. [16] reviewed retail forecasting research and practice and stressed store-level granularity, promotional effects and the gap between academic methods and operational use.

2.2.5 Explainability

For forecasts to be trusted by planners, they need to be explainable. SHAP [15] attributes each prediction to the contributions of individual features using Shapley values from cooperative game theory, and provides both global importance rankings and per-prediction explanations. Its TreeExplainer variant computes exact values efficiently for tree ensembles [21].

2.3 Comparison of Existing Methods

Table 1 summarises the main families of methods against the requirements of this project.

Table 1: Comparison of forecasting method families

Method family	Strengths	Limitations for this problem	Examples
Naive / moving average	No training; transparent; strong weekly benchmark	Cannot react to promotions or holidays	Seasonal-naive, 7-day mean
Exponential smoothing / ARIMA	Well understood; good for smooth single series	One model per store; few external drivers	[1], [2]
Decomposition models	Interpretable trend and seasonality; holiday terms	Limited cross-store learning	STL [3], Prophet [4]
Linear regression	Fast; interpretable coefficients	Misses non-linear interactions	Ridge [24]
Gradient-boosted trees	Non-linear; many drivers; global models; fast	Needs careful leakage-free features	XGBoost [6], LightGBM [5]
Deep learning	Learns representations; probabilistic output	Heavier tuning and compute; mixed gains on tabular data	DeepAR [11], embeddings [10]

2.4 Identified Research Gaps

The review suggests that gradient boosting with engineered features is the most appropriate method family for this problem. It also reveals gaps that this project sets out to address:

1. **Leakage is usually asserted, not tested.** Many published notebooks and competition solutions state that features are leakage-free, but few include an executable check that independently recomputes features from raw history.
2. **Evaluation often relies on a single split.** A single hold-out period can be unrepresentative. Multi-fold walk-forward evaluation, scored with a scale-free measure such as MASE, gives a more reliable picture [12], [13].
3. **Results are rarely runnable.** Forecasting studies are typically reported as static tables and charts. There are few examples of a pipeline in which a reader can re-run forecasts, explanations and validation themselves.
4. **Cross-retailer transfer is under-explored.** Global models are known to benefit from more related series [22], but whether data from a **different retailer**, with a different currency, region and time period, helps is rarely tested directly.

2.5 Chapter Summary

Classical methods provide strong, interpretable benchmarks but scale poorly to many stores with many external drivers. Gradient-boosted trees, trained as a single global model on lag, rolling and calendar features, are the leading practical approach for store-level retail forecast-

ing. Correct evaluation requires walk-forward validation, leakage-free features and scale-free error measures. This project adopts that approach and addresses the four gaps above through executable leakage checks, five-fold walk-forward evaluation, an interactive application and a cross-retailer experiment.

Chapter 3: Research Methodology

3.1 Research Design

The project follows a quantitative, experimental research design. Competing forecasting models are trained on historical data and compared on identical, strictly out-of-time validation periods using pre-defined error metrics. The work proceeds through ten sequential stages, each implemented as a numbered Jupyter notebook that saves its output for the next stage (Table 2).

Table 2: Project stages and corresponding notebooks

Stage	Notebook	Purpose
00	Data acquisition and overview	Load and type the raw data; initial inspection
01	Data cleaning and preprocessing	Merge store attributes; encode holidays; impute missing fields
02	Exploratory data analysis	STL decomposition; seasonality; promotion and store-type effects
03	Feature engineering	Lag, rolling and calendar features; leakage check
04	Baseline and linear models	Seasonal-naive, moving-average and Ridge regression
05	Gradient boosting model	LightGBM on log-transformed sales
06	Walk-forward validation	Five-fold comparison of all models
07	Model interpretation	Feature importance and SHAP analysis
08	Pipeline summary	48-day forecast on the real test window
09	Multi-retailer experiment	Rossmann + Walmart weekly ablation

Shared logic (cleaning, features, metrics, validation splits and model builders) lives in a tested Python package, `src/`, which every notebook imports. As a result, the training, validation, forecasting and web-application code paths all use the same functions.

3.2 Dataset / Data Sources

Two real, publicly available Kaggle datasets were used (Table 3). The primary dataset is Rossmann Store Sales [19], which consists of three files: `train.csv` (daily sales history), `store.csv` (one row of attributes per store) and `test.csv` (the 48-day period to be forecast, without sales values). The secondary dataset, Walmart Sales [20], was used only for the multi-retailer experiment.

Table 3: Summary of the datasets used

	Rossmann Store Sales (primary)	**Walmart Sales (experiment)**
Grain	Store × day	Store × week
Stores	1,115	45
Rows	1,017,209	6,435
Date range	1 Jan 2013 – 31 Jul 2015	5 Feb 2010 – 26 Oct 2012
Target	`Sales`	`Weekly_Sales`
Other key fields	Customers, Open, Promo, StateHoliday, SchoolHoliday, StoreType, Assortment, CompetitionDistance, Promo2	Holiday_Flag, Temperature, Fuel_Price, CPI, Unemployment
Held-out test set	41,088 rows: 856 stores × 48 days (1 Aug – 17 Sep 2015), no sales values	None

Key characteristics of the Rossmann training data:

- **Closed days.** 172,817 rows (17.0%) are days on which a store was closed (`Open` = 0) and sales are zero. A further 54 rows record zero sales on an open day.
- **Store types.** Stores belong to four types: a (602 stores), d (348), c (148) and b (17). Type b stores have much higher average sales (10,231 per open day) than the other types (6,822–6,933).
- **Holidays.** 31,050 rows fall on a state holiday: 20,260 public holidays (a), 6,690 Easter (b) and 4,100 Christmas (c).
- **Average demand.** Mean sales on open days are 6,955.5, with a median of 6,369.
- **Missing values in store attributes.** `CompetitionDistance` is missing for 3 stores, the competitor opening date for 354 stores, and the Promo2 start date and interval for 544 stores (stores that do not take part in Promo2).

The Walmart and Rossmann datasets share no stores and no dates; the Walmart period ends about two months before the Rossmann period begins. This separation is important for the experiment in Section 5.3.

3.3 Data Collection and Preprocessing

The data was downloaded from Kaggle through the author's own account, as both datasets require accepting Kaggle's terms; for the same reason, the data files are not redistributed in the project repository. Preprocessing is implemented in `src/cleaning.py` as a single function, `clean_pipeline`, which is applied identically to the training and test data:

1. **Merge.** Store attributes are joined onto every sales row. The merge asserts that the row count is unchanged.
2. **Sort.** Rows are sorted by store and date, so that each store's history is in time order.

3. **Closed days kept.** Rows with `Open` = 0 are deliberately kept rather than dropped. Removing them would break each store's continuous daily calendar, so a "7-day lag" would no longer mean seven calendar days. The row count is preserved throughout cleaning: 1,017,209 rows in, 1,017,209 rows out.
4. **Missing `Open` flags.** A small number of test rows lack an `Open` value; they are treated as open.
5. **Holiday encoding.** `StateHoliday` is normalised to a consistent category (none, a, b, c) and a binary `is_state_holiday` flag is derived.
6. **Competition fields.** Missing `CompetitionDistance` values are filled with the median and recorded in a `competition_distance_missing` flag. A `has_competition_since_info` flag records whether the competitor opening date is known. From the opening date the pipeline derives `competition_open_days` and `has_competitor_open`, both computed relative to each row's own date so that they never use future information.
7. **Promo2.** A `promo2_active` flag is decoded for each row from the store's Promo2 start week and its `PromoInterval` months (for example, "Feb,May,Aug,Nov").

Missing values are thus never silently guessed: each imputation is paired with an explicit indicator so the model can tell real values from filled ones.

Exclusion of `Customers`. The number of customers is strongly correlated with sales (Pearson r = 0.895 over all rows, 0.824 on open days). However, it is only known at the end of the day being forecast, and it is absent from the real test set. Using it would be a textbook case of target leakage [18], so it is excluded from all models.

3.4 Proposed Methods / Algorithms

3.4.1 Exploratory analysis

Total daily chain-wide sales were decomposed using STL [3] with a period of 7 days, separating trend, weekly seasonality and residual. Day-of-week profiles, promotion lift, store-type distributions and correlations were examined to guide feature design.

3.4.2 Feature engineering

Features are built in `src/features.py`. There are 37 model inputs in five groups (Table 4).

Table 4: Model input features

Group	Features	Count
Lag features	`lag_1`, `lag_7`, `lag_14`, `lag_28`: sales 1, 7, 14 and 28 days earlier	4
Rolling statistics	Mean and standard deviation over 7, 14 and 28 days; minimum and maximum over 7 days; number of promotion days in the previous 7 days	9
Calendar	Year, Month, Day, WeekOfYear, Quarter, weekend, month start/end, day before/after a state holiday	10
Store, promotion, competition	CompetitionDistance and its missing flag, competition open days, competitor-open flags, Promo, SchoolHoliday, state-holiday flag, promo2_active	9
Categorical	Store, StoreType, Assortment, state-holiday type, DayOfWeek	5
Total		**37**

Leakage-safe construction. For every lag and rolling feature, the sales series is first shifted back by one day **within each store**, and only then is the window computed. As a result, the feature for a given store and date only uses that store's strictly earlier sales, and a window never crosses from one store into another. Windows require complete history (`min_periods` equals the window length), so the first 28 days of each store's history have no features and are not used for training.

Executable leakage check. The function `assert_no_leakage()` selects 30 random (store, date) pairs, recomputes `lag_7` and `roll_mean_7` from scratch using only raw rows strictly earlier than that date, and asserts that they match the stored feature values. The check was run on a synthetic rehearsal dataset and on the full 1,017,209-row real dataset, and passed both times.

Trainable rows. A row is used for training only if the store was open and all lag and rolling features are available. This leaves 818,845 trainable rows (80.5% of the data). The excluded rows are closed-store days plus the start-up period of each store's history.

3.4.3 Forecasting models

Four models were compared, ranging from no training at all to gradient boosting:

1. **Seasonal-naive.** Tomorrow's sales are predicted to equal sales on the same weekday one week earlier (`lag_7`). This is the standard benchmark for series with weekly seasonality [1].
2. **Moving average.** The prediction is the mean of the previous 7 days (`roll_mean_7`).
3. **Ridge regression** [24]. A linear model with L2 regularisation ($\alpha = 1.0$). Numeric features are standardised and categorical features one-hot encoded in a scikit-learn pipeline [17].
4. **LightGBM** [5]. A gradient-boosted decision-tree model trained on all 37 features. The main settings are 63 leaves per tree, a learning rate of 0.05, up to 1,000 boosting rounds with early stopping after 50 rounds without improvement, and 80% row and column subsampling. All categorical variables, including the 1,115-value `Store` column, are passed as native categories rather than one-hot encoded.

Both trained models (Ridge and LightGBM) are fitted to $\log(1 + \text{Sales})$ rather than raw sales, and predictions are transformed back with $e^x - 1$. The log transform stabilises the variance across small and large stores and reduces the influence of extreme days.

Global model. A single model is trained across all 1,115 stores rather than one model per store. This follows the evidence that global models generalise well across related series [22] and lets stores with short or noisy histories borrow strength from the others.

3.4.4 Validation scheme

Models are evaluated with five-fold expanding-window walk-forward validation, implemented in `src/validation.py`. The five 42-day validation blocks are laid end to end, working backwards from the last date in the data. For each fold, the model is trained on every date before the validation block and scored on that block (Table 5). Forty-two days was chosen to match the length of the real forecasting task. No shuffling is performed anywhere: every split is a strict date cut, and this is asserted in code.

Table 5: Training and validation sizes for each walk-forward fold

Fold	Training rows	Validation rows
1	623,946	39,924
2	663,870	40,176
3	704,046	37,044
4	741,090	37,473
5	778,563	40,282

3.4.5 Interpretation and forecasting

Model behaviour is explained with LightGBM's gain-based feature importances and with SHAP values computed by TreeExplainer [15], [21]. For the final forecast, the same cleaning and feature functions are applied unchanged to the real test window, and the model forecasts recursively, one day at a time, as described in Section 4.3.

3.5 Tools, Software, and Experimental Setup

Table 6: Software tools and libraries

Tool / library	Version	Use in this project
Python	3.11.15	Implementation language
pandas	3.0.6	Data loading, cleaning and feature engineering
scikit-learn	1.9.1	Ridge regression, preprocessing pipeline
LightGBM	4.7.0	Gradient-boosting model
SHAP	0.51.0	Model explanations
statsmodels	0.15.0	STL decomposition
matplotlib	3.11.2	Figures
FastAPI	0.156.8	Interactive web application
pytest	—	Unit tests (22 tests)
Jupyter	—	Notebook-based pipeline stages
Git / GitHub	—	Version control and code publication

All experiments were run on a single laptop: an Apple M4 Pro processor with 24 GB of memory, running macOS. A fixed random seed (42) is used for every model that involves randomness, so the results are reproducible.

3.6 Evaluation Metrics

Let y_t be the actual sales, $\hat{y}_t$ the forecast and n the number of validation rows. Four metrics are reported.

Root Mean Squared Error (RMSE) penalises large errors heavily:

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{t=1}^{n}\left(y_t - \hat{y}_t\right)^2}$$

Mean Absolute Error (MAE) gives the average size of an error in sales units:

$$\mathrm{MAE} = \frac{1}{n}\sum_{t=1}^{n}|y_t - \hat{y}_t|$$

Weighted Mean Absolute Percentage Error (WMAPE) expresses total absolute error as a share of total sales. Unlike ordinary MAPE, it remains well defined when individual actual values are zero:

$$\mathrm{WMAPE} = \frac{\sum_t |y_t - \hat{y}_t|}{\sum_t |y_t|}$$

Mean Absolute Scaled Error (MASE) [12] divides the model's MAE by the in-sample MAE of a seasonal-naive forecast with period $m = 7$ over the T training observations:

$$\text{MASE} = \frac{\text{MAE}}{\frac{1}{T-m}\sum_{t=m+1}^{T}|y_t - y_{t-m}|}$$

A MASE below 1 means the model beats the seasonal-naive benchmark. Because a single model is trained across all stores, the naive differences in the denominator are computed **within each store** and then pooled (the `train_groups` argument of the implementation). This prevents a difference being taken across the boundary between two stores in the concatenated training data; a dedicated unit test verifies the behaviour.

WMAPE was chosen as the primary comparison metric because it is scale-free and easy to interpret. The acceptance criterion defined in Section 1.5 uses WMAPE and MASE.

3.7 Ethical Considerations

The datasets contain aggregated store-level sales and contain no personal or customer-identifying information. Both were obtained from Kaggle under their published terms of use, which is why the raw files are not redistributed with the code. No human participants were involved. All results are reported as produced by the code, including the limitations and the one fold in which the multi-retailer model performed slightly worse.

Chapter 4: System Design and Implementation

4.1 System Architecture / Proposed Framework

The system is organised into four layers (Figure 1): a data layer of raw and intermediate files, a tested pipeline library, a modelling layer of notebooks and saved models, and a serving layer that exposes the models through a web application.

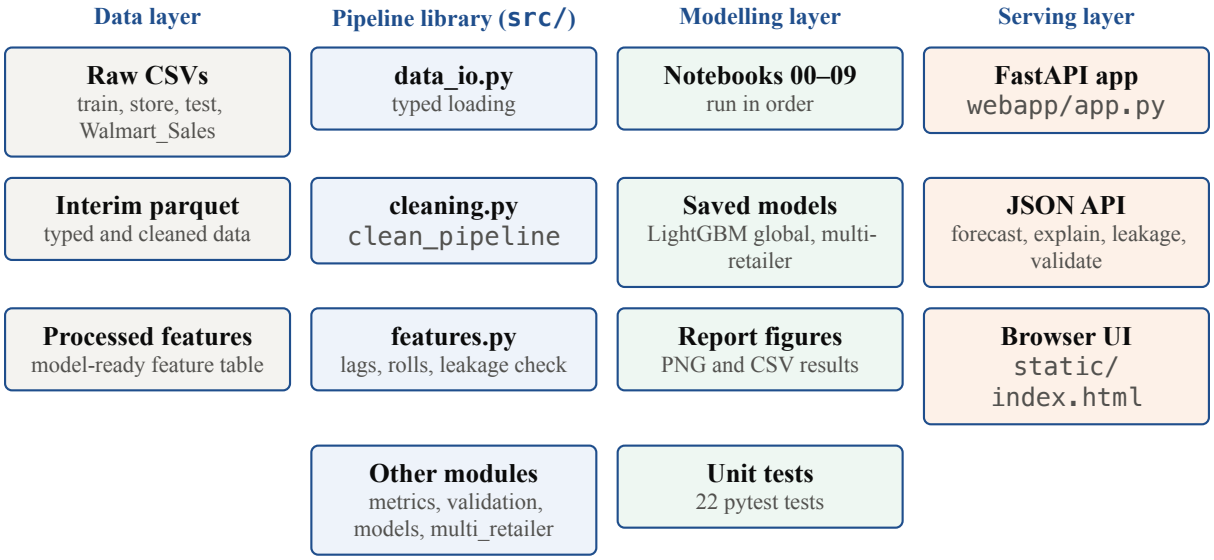


Figure 1: System architecture

The central design decision is that **all** logic used to transform data or score models lives in one place, the `src/` package, and is reused unchanged everywhere. The notebooks, the forecasting step and the web application all call the same `clean_pipeline` and `build_feature_matrix` functions. This prevents a common failure in deployed forecasting systems, where features at prediction time are computed slightly differently from features at training time.

4.2 Workflow or Process Diagram

Figure 2 shows the end-to-end workflow from raw data to forecast.

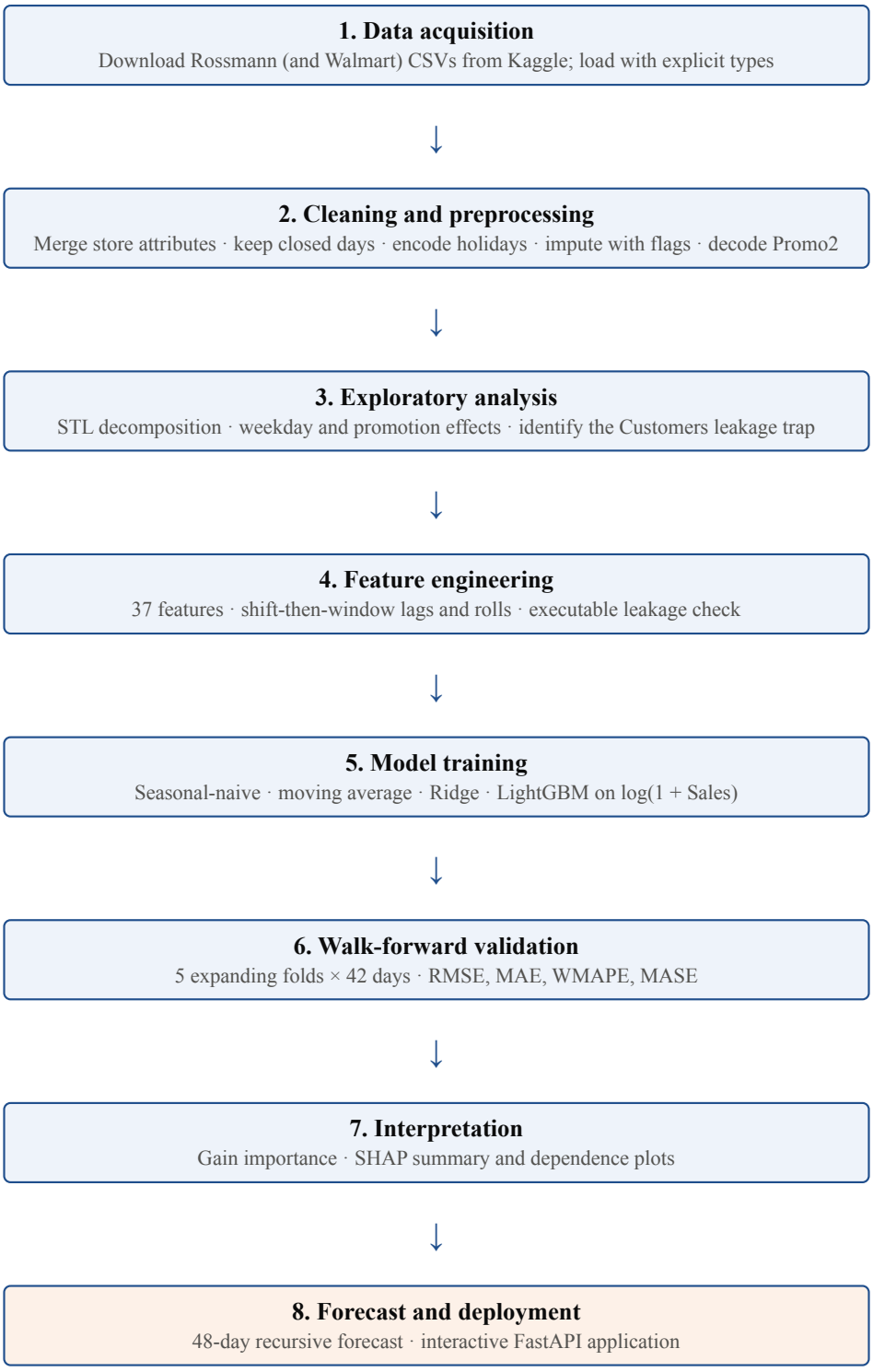


Figure 2: End-to-end project workflow

4.3 Model / System Design

4.3.1 Global LightGBM model

The final model is a single LightGBM regressor over all stores, with `Store` as a native categorical feature. Treating store identity as a category lets the trees learn each store's level directly while sharing calendar, promotion and momentum effects across all stores. One-hot

encoding 1,115 stores would create 1,115 sparse columns; the native categorical split handles this efficiently.

4.3.2 Recursive multi-step forecasting

The real test period runs 48 days beyond the last known sales date, but the lag and rolling features need up to 28 days of recent sales. After the first day, some of that "recent history" consists of days that have not happened yet. The system therefore uses the standard recursive strategy [23]:

1. Predict the first forecast day using actual history.
2. Append that prediction to the store's history as if it were an observed value.
3. Recompute the lag and rolling features for the next day and predict again.
4. Repeat until the 48-day horizon is complete. Days on which the store is scheduled to be closed are forecast as zero.

This approach uses one model for every horizon, but errors can compound as more of the feature window is filled with predictions rather than actual values, a limitation discussed in Section 5.5.

4.3.3 Multi-retailer unified schema

For the multi-retailer experiment, `src/multi_retailer.py` converts both datasets to a common store-week schema. Rossmann's daily data is aggregated **up** to weekly, rather than Walmart's weekly data being split **down** to daily, because inventing daily values from a weekly total would mean training on data that does not exist. Both retailers number their stores from 1, so every store is given a dataset-prefixed identifier (for example `rossmann_262` and `walmart_12`) to prevent collisions. A `dataset_source` feature tells the model which retailer a row comes from. Walmart has no promotion data, so its promotion field is left missing rather than filled with a guess. The same leakage check, adapted to weekly grain, passed on the combined dataset of 152,460 store-weeks.

4.4 Implementation Details

4.4.1 Repository structure

Table 7: Repository structure

Directory	Contents
`notebooks/`	Ten notebooks: 00–08 form the main pipeline; 09 is the multi-retailer experiment
`src/`	`data_io`, `cleaning`, `features`, `metrics`, `validation`, `models`, `multi_retailer`, `viz`
`scripts/`	`train_multi_retailer.py`, a standalone script version of notebook 09
`webapp/`	`app.py` (FastAPI server) and `static/index.html` (browser interface)
`tests/`	22 pytest unit tests for metrics, features and the multi-retailer schema
`data/`	`raw/`, `interim/` and `processed/` (data files are not committed)
`models/`	Saved model files (not committed)
`reports/figures/`	Generated figures and result tables

4.4.2 Testing

The 22 unit tests in `tests/` check the core library against small synthetic series with hand-computed expected values. They cover every metric (including the per-store MASE scaling), the lag and rolling features (including that windows do not cross store boundaries and only use earlier dates), the leakage check itself, and the multi-retailer schema (including store-identifier prefixing). The tests need no Kaggle data and run in seconds.

4.4.3 Interactive web application

`webapp/app.py` is a FastAPI application. At start-up it loads the real data and both saved models into memory. It then serves a single-page browser interface whose panels call live API routes. Every route re-runs project code on each request instead of reading precomputed results (Table 8).

Table 8: Web application panels and API routes

Panel	Route	What it does	Measured time
Live forecast	`GET /api/forecast`	Runs the recursive forecast for any of the 856 test stores and a 1–48 day horizon	93–280 ms
Explain this day	`GET /api/explain`	Computes the SHAP breakdown of one forecast day	43–55 ms
Leakage check	`GET /api/leakage-check`	Re-verifies a fresh random sample of features against raw history	8–30 ms
Re-run validation	`POST /api/validate`	Retrains Ridge and LightGBM across all five folds; cached after the first run	about 100–140 s
Multi-retailer	`/api/multi-retailer/...`	Weekly prediction for any store from either retailer; on-demand re-run of the experiment	4–6 ms / about 140 s

One subtle correctness issue was found and fixed during implementation. If a small slice of data, such as a single store or a single row, is converted to the pandas `category` type on its own, pandas numbers the categories using only the values present in that slice. The numbering then no longer matches the one the model learned in training, which silently corrupts both predictions and SHAP explanations. The application fixes each categorical column's full set of levels once at start-up (`CATEGORY_LEVELS`) and applies that exact mapping to every request, however small.

4.5 Computational Requirements

Table 9: Computational requirements

Task	**Requirement on the test machine**
Hardware used	Apple M4 Pro, 24 GB RAM; no GPU required
Five-fold walk-forward validation (Ridge + LightGBM)	About 100–140 seconds
Web application start-up	10–30 seconds to load and clean the full dataset
Single-store 48-day forecast	Under 0.3 seconds
Unit test suite	A few seconds; no dataset needed

These modest requirements mean the entire project, including retraining, can be reproduced on an ordinary laptop.

Chapter 5: Results and Discussion

5.1 Experimental Results

5.1.1 Exploratory analysis findings

The STL decomposition (Figure 3) shows a strong, stable weekly cycle, a trend that rises sharply each December before Christmas, and a residual containing sharp downward spikes on public holidays when most stores close.

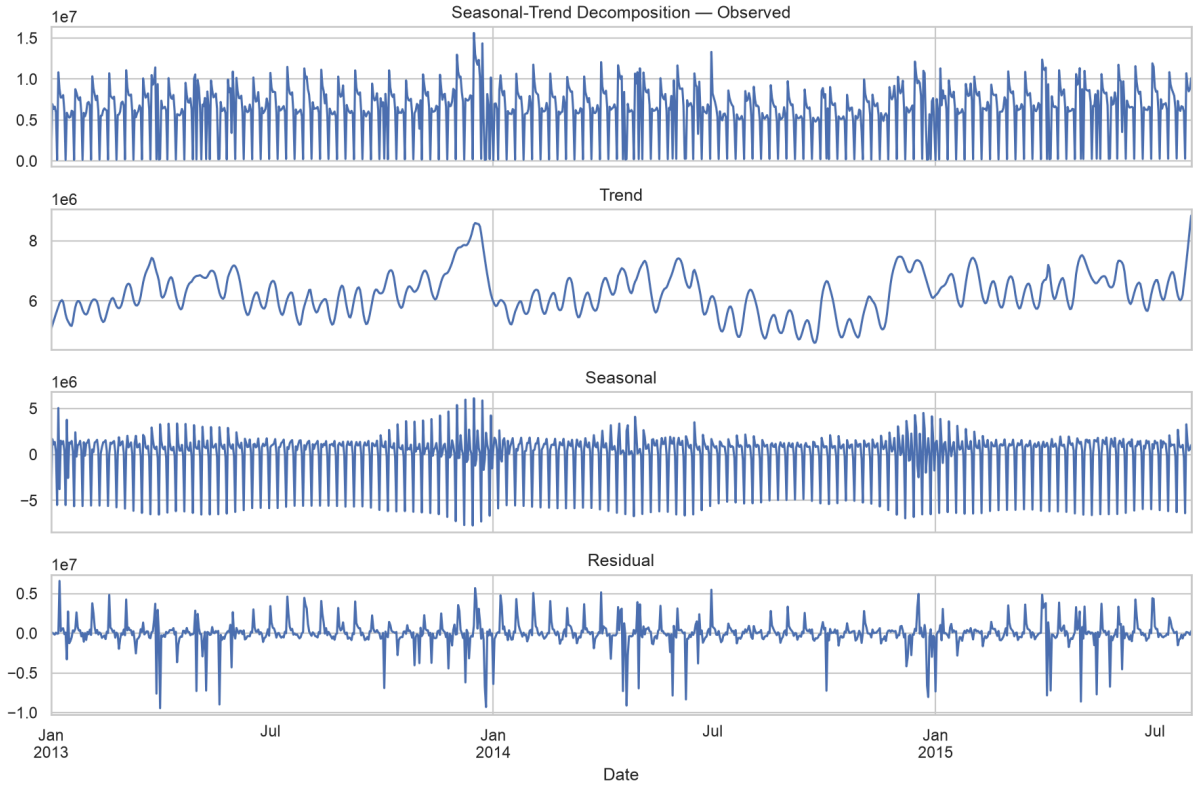


Figure 3: STL decomposition of total daily sales into trend, weekly seasonal and residual components

Average sales on open days vary strongly by weekday (Figure 4): Monday (8,216) and Sunday (8,225, when only a few stores open) are the highest, and Saturday (5,875) is the lowest. Promotions have a large effect (Figure 5): mean sales on promotion days are 8,228 compared with 5,929 on other days, a lift of 38.8%. Store type b has far higher average sales than the other types (Figure 6).

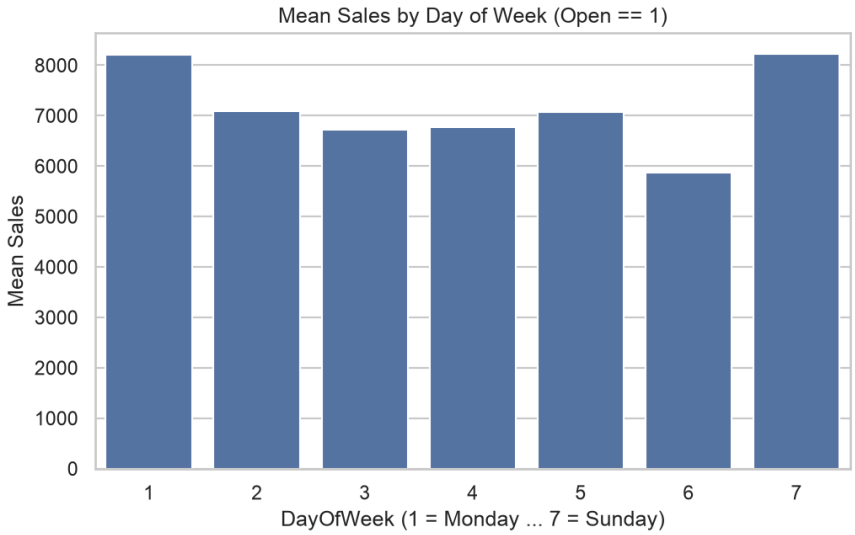


Figure 4: Mean sales by day of the week

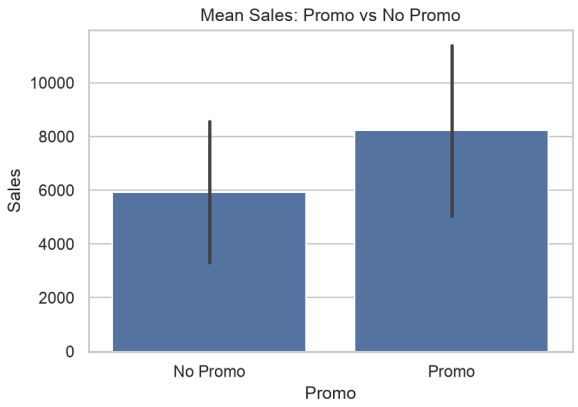


Figure 5: Mean sales with and without a promotion

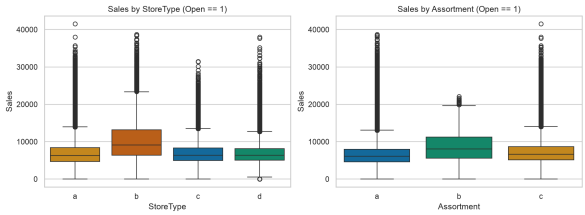


Figure 6: Sales distribution by store type

The analysis also identified `Customers` as a leakage trap: it is the variable most correlated with sales (r = 0.895), but it is not known in advance and does not exist in the test data. It was therefore excluded, as described in Section 3.3.

5.1.2 Walk-forward validation results

Table 10 reports the mean of each metric across the five validation folds. LightGBM is the best model on every metric.

Table 10: Mean validation error across five walk-forward folds (lower is better)

Model	RMSE	MAE	WMAPE	MASE
Seasonal-naive	3,011	2,229	31.19%	0.956
Moving average	2,378	1,719	24.09%	0.737
Ridge regression	1,264	845	11.87%	0.362
LightGBM	**886**	**560**	**7.87%**	**0.240**

Figure 7 and Table 11 show WMAPE for every model in every fold. LightGBM has the lowest WMAPE in all five folds, ranging from 7.06% (fold 2) to 8.64% (fold 1).

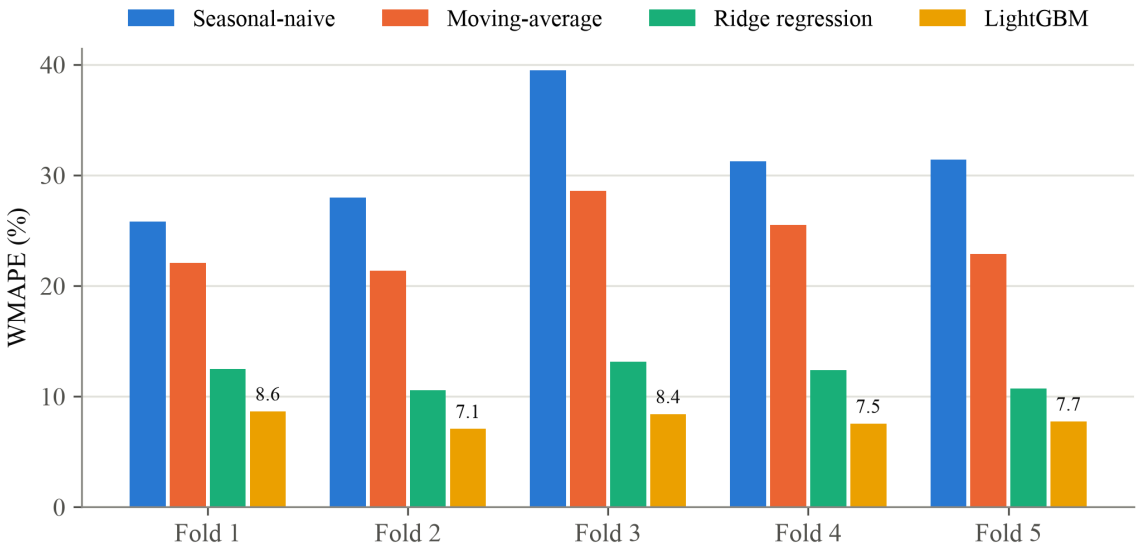


Figure 7: WMAPE by model and validation fold; labels show the LightGBM value

Table 11: WMAPE by fold for each model

Fold	Seasonal-naive	Moving average	Ridge	LightGBM
1	25.82%	22.08%	12.50%	**8.64%**
2	27.97%	21.36%	10.55%	**7.06%**
3	39.49%	28.61%	13.15%	**8.40%**
4	31.28%	25.50%	12.41%	**7.53%**
5	31.41%	22.89%	10.72%	**7.73%**

5.1.3 Forecasting demonstration

Applying the trained model to the real Kaggle test window produced 41,088 forecasts: 856 stores × 48 days, from 1 August to 17 September 2015. Every forecast is a valid number: there are no missing and no negative predictions. (Kaggle's test set covers 856 of the 1,115 stores; this is a property of the competition data, not an error.) Figure 8 shows total forecast demand following on from the last eight weeks of actual sales. The forecast reproduces the weekly cycle and the near-zero Sundays seen in the history.

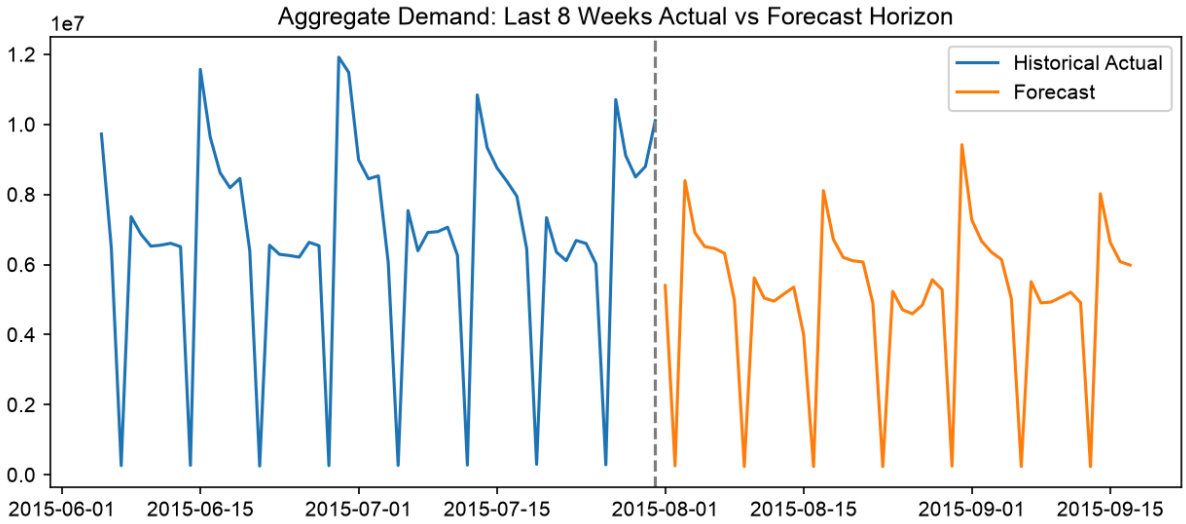


Figure 8: Total chain demand: last eight weeks of actual sales and the 48-day forecast

5.2 Quantitative and Qualitative Evaluation

Quantitative. Each increase in model capability roughly halves the error: WMAPE falls from 31.2% (seasonal-naive) to 24.1% (moving average), 11.9% (Ridge) and 7.9% (LightGBM). LightGBM's MASE of 0.240 means its typical error is about a quarter of the seasonal-naive error. Its MAE of 560 sales units per store-day compares with mean open-day sales of about 6,956. The acceptance criterion from Section 1.5 is comfortably met: LightGBM beats every baseline on WMAPE in 5 of 5 folds (criterion: at least 4), and its mean MASE of 0.240 is well below 1.0.

LightGBM is also consistent. Its WMAPE varies by only 1.6 percentage points across folds, while the seasonal-naive WMAPE varies by 13.7 points; fold 3 is the hardest for the baselines.

Qualitative. Figure 9 compares LightGBM's predictions with actual sales for three sample stores over the final validation period. The model follows the weekly pattern, the promotion-driven peaks and the troughs closely. Its largest misses are on the most extreme peak days, where it tends to under-predict slightly. This is typical of a model trained on log-scale errors, which treats a given proportional error the same at any sales level.

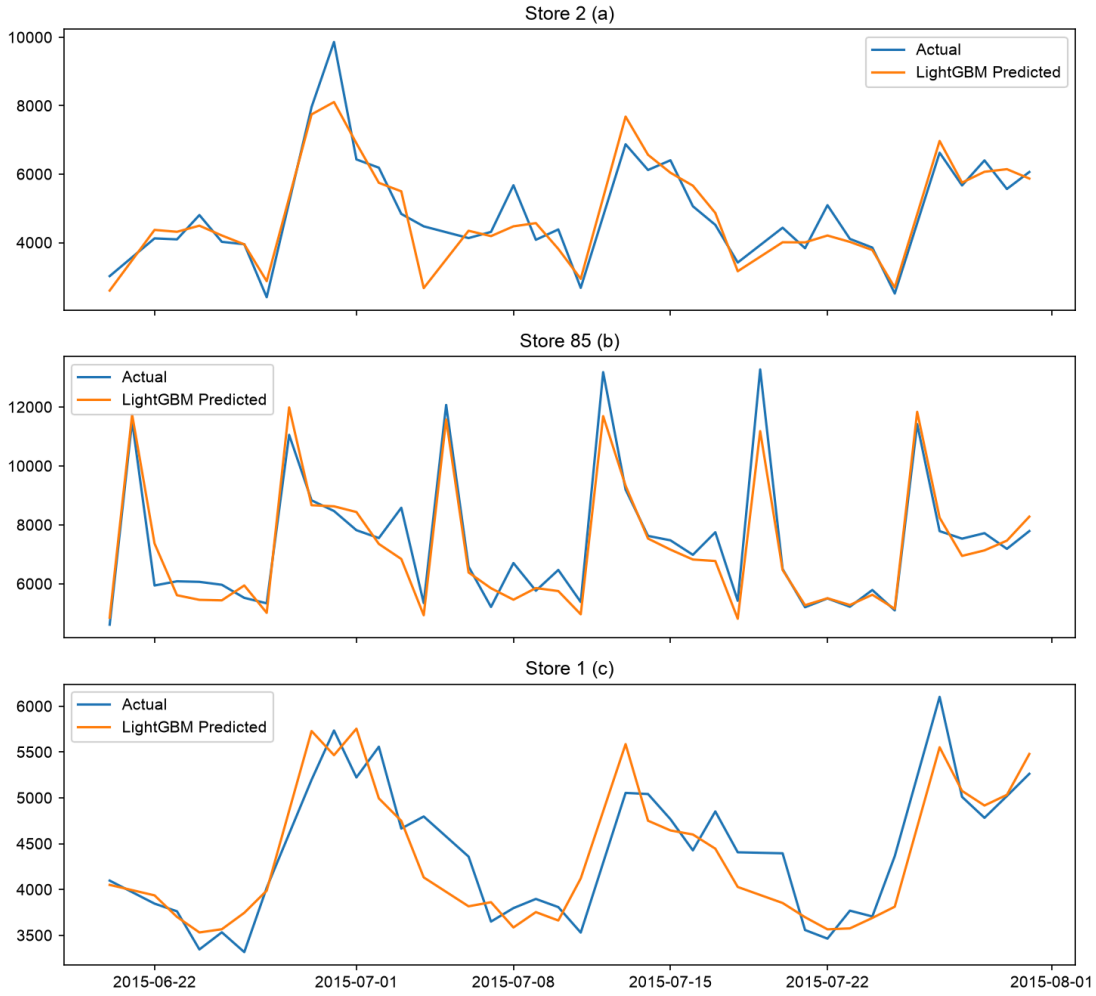


Figure 9: LightGBM predictions against actual sales for three sample stores

5.3 Comparison with Baseline / Existing Methods

5.3.1 Comparison with baselines

The two naive baselines represent what a planner might do without a model. Ridge regression already cuts the seasonal-naive error by more than half, which shows the value of the engineered features alone. LightGBM cuts the remaining error by a further third (11.87% to 7.87% WMAPE) by capturing non-linear interactions, for example between promotions, weekday and store type, that a linear model cannot represent. This ordering matches the broader evidence from forecasting competitions that gradient-boosted trees outperform linear and statistical models on this kind of data [8], [9].

5.3.2 Multi-retailer experiment

Notebook 09 tests RQ4: does training on Walmart's sales history improve Rossmann's own forecasts? Two weekly LightGBM models were compared on the **same** five Rossmann validation weeks. The first was trained on Rossmann data only; the second was trained on Rossmann plus Walmart. Because every Walmart date precedes every Rossmann date, Walmart rows are

always in the training window and never in a validation fold. They can only add training signal, so the size of the training data is the only difference between the two models.

Table 12: Multi-retailer experiment: Rossmann weekly validation error

Fold	WMAPE Rossmann only	WMAPE combined	MASE Rossmann only	MASE combined
1	9.45%	9.60%	0.334	0.251
2	5.15%	4.99%	0.182	0.131
3	5.63%	5.38%	0.210	0.152
4	5.35%	5.34%	0.190	0.146
5	9.96%	9.75%	0.357	0.271
Mean	**7.11%**	**7.01%**	**0.255**	**0.190**

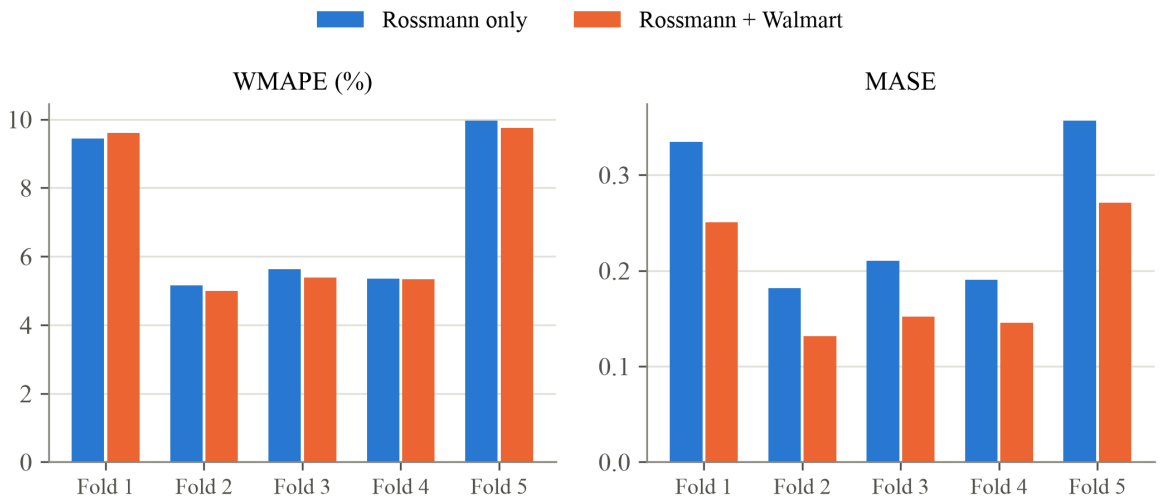


Figure 10: Rossmann-only versus combined (Rossmann + Walmart) training, by fold

The combined model achieved lower WMAPE in 4 of 5 folds and lower MASE in all 5, improving mean MASE by about 25% relative (0.255 to 0.190). The WMAPE gain is small (0.10 percentage points) while the MASE gain is large. This suggests that Walmart's data mainly helps on the store-weeks where week-over-week persistence is a poor guide, which MASE weights more heavily. Note that MASE here uses a one-week naive baseline (week-over-week persistence), not the 7-day seasonal baseline used for the daily model, so these values cannot be compared directly with Section 5.1.

5.4 Interpretation of Findings

Figure 11 shows LightGBM's gain-based feature importance. Store identity dominates (`Store`, 33.4% of total gain), which is expected for a chain with large differences between stores. It is followed by calendar position (`Day` 7.1%, `WeekOfYear` 5.2%, `DayOfWeek` 4.9%) and recent sales momentum (`lag_1` 4.3%, `lag_14` 4.2%, `roll_mean_7` 3.4%).

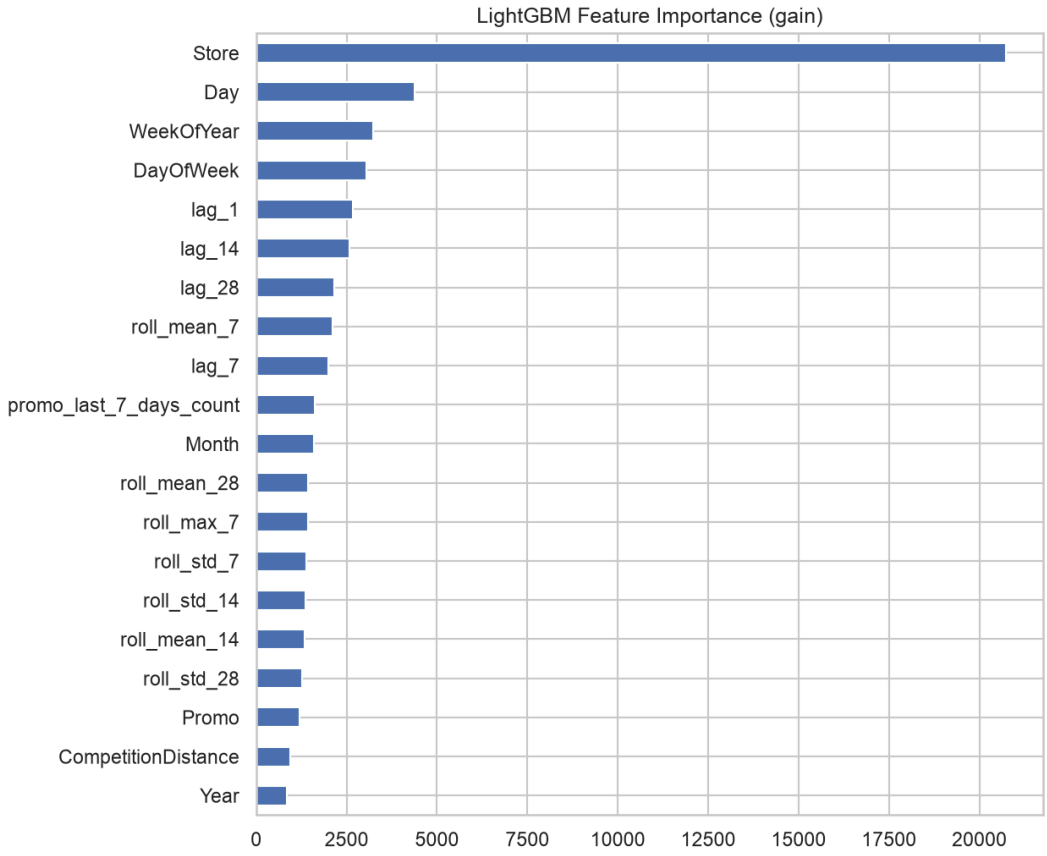


Figure 11: LightGBM feature importance (gain)

The SHAP summary (Figure 12) confirms this ranking and adds the **direction** of each effect. An active promotion consistently pushes predictions upward, by a large amount. State holidays push predictions down. `CompetitionDistance` has a real but much smaller and more gradual effect.

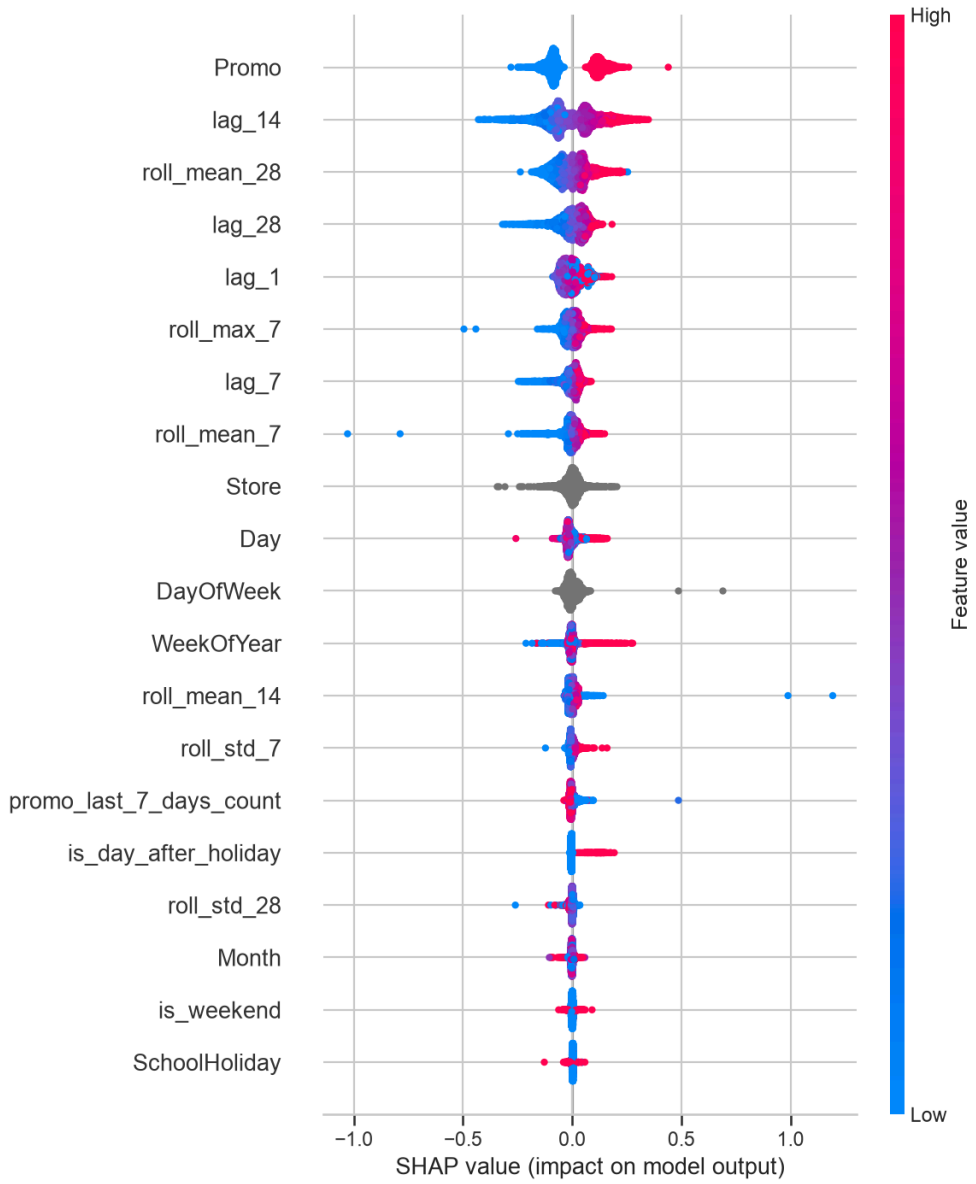


Figure 12: SHAP summary of feature effects on individual predictions

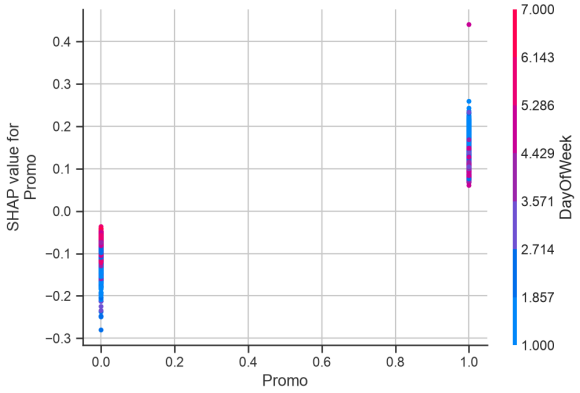


Figure 13: SHAP dependence: Promo

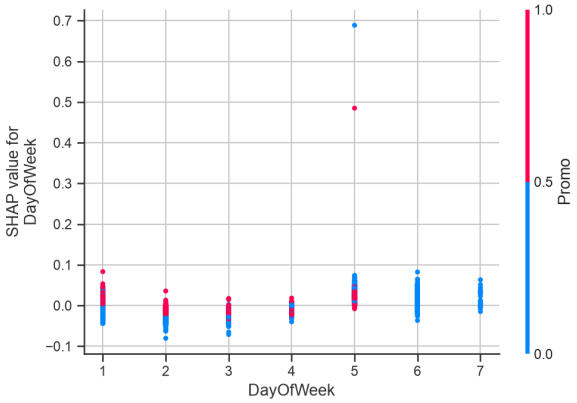


Figure 14: SHAP dependence: DayOfWeek

In answer to the research questions:

- **RQ1** is answered positively: LightGBM is clearly and consistently more accurate than the baselines and Ridge regression.
- **RQ2:** the main drivers of demand are store identity, calendar position, recent momentum and promotions. Competition has a smaller effect.
- **RQ3:** the model produces a complete, valid 48-day forecast on genuinely unseen dates.
- **RQ4:** adding a second retailer's history gives a small but consistent improvement, which suggests that seasonal and holiday structure transfers between retailers even with no overlap in stores, dates or currency.

5.5 Limitations and Threats to Validity

Internal validity.

- **Early stopping on the validation fold.** In the walk-forward evaluation, LightGBM's early stopping monitors the same 42-day block on which the fold is then scored. This lets the validation data choose the number of boosting rounds, which biases the reported LightGBM scores slightly in its favour. The bias is expected to be small, because only one setting is chosen this way and the margin over Ridge is large (7.87% against 11.87%), but a strictly clean protocol would use a separate early-stopping period inside each training window.
- **No hyperparameter search.** The LightGBM settings are reasonable defaults rather than tuned values. This makes the comparison conservative, but means the reported accuracy may not be the best achievable.

Construct validity.

- **Recursive error accumulation.** In the 48-day forecast, prediction errors can compound once the 28-day feature window is filled with earlier predictions. The test set has no actual sales, so forecast accuracy over the full horizon could not be measured directly; the validation results are the best available estimate.
- **Promo2 approximation.** Promo2 activity is decoded from `PromoInterval` and is an approximation of the true promotional calendar.

External validity.

- **Single retailer for the main model.** The daily model was developed and validated on one retailer in one country. The multi-retailer experiment is a first test of generalisation, at weekly grain only.
- **No currency or scale alignment** between retailers in the experiment. The model relies on store and dataset identifiers to separate the two regimes, and no numeric equivalence between them is assumed.

- **No external data.** Weather, local events and economic conditions are not used by the daily model.

Chapter 6: Conclusion and Future Work

6.1 Summary of the Research

This project set out to replace manual, average-based demand estimates with an accurate, data-driven forecasting system for individual retail stores. Using 1,017,209 real daily records from 1,115 Rossmann stores, it built a complete pipeline: leakage-safe cleaning that preserves every row, exploratory decomposition of trend and seasonality, 37 engineered features verified by an executable leakage check, four competing models, five-fold walk-forward validation, SHAP-based interpretation, and a 48-day forecast on the real held-out test window. The pipeline was then extended into an interactive web application and tested for generalisation with data from a second retailer.

6.2 Major Findings

1. A single global LightGBM model achieved a mean WMAPE of 7.87% and MASE of 0.240, beating seasonal-naive (31.19%), moving-average (24.09%) and Ridge regression (11.87%) baselines in every one of the five validation folds.
2. Each step up in model capability roughly halved the error, and LightGBM's performance was stable across folds (7.06%–8.64% WMAPE).
3. Store identity, calendar position, recent sales momentum and promotions are the main drivers of demand. Promotions raise average sales by 38.8%.
4. The model produced a complete and valid 48-day forecast for all 856 test stores, with no missing or negative values.
5. Adding Walmart's sales history improved Rossmann's weekly forecasts (mean MASE 0.255 to 0.190; WMAPE 7.11% to 7.01%), even though the two datasets share no stores, dates or currency.

6.3 Research Contributions

- **A verified leakage-safe pipeline.** Features are leakage-free by construction (shift before windowing, within each store), and this is confirmed by an independent recomputation check on the full dataset, which also runs live in the application.
- **Rigorous, reproducible evaluation.** Five-fold walk-forward validation with a store-aware MASE, an explicit acceptance criterion set before modelling, fixed random seeds and 22 unit tests.
- **A runnable rather than static result.** An interactive web application lets anyone re-run forecasts, explanations, leakage checks and the full validation on demand.

- **Evidence on cross-retailer transfer.** A controlled experiment showing that a different retailer's history can improve forecasts, using a unified weekly schema that avoids fabricating data or merging store identifiers.

6.4 Limitations

The main limitations, discussed in detail in Section 5.5, are:

- early stopping uses the validation fold, which slightly flatters LightGBM;
- hyperparameters were not tuned;
- recursive forecasts may accumulate error over long horizons;
- no external data such as weather or local events is used;
- the daily model is validated on a single retailer;
- the cross-retailer experiment is limited to weekly grain and two retailers.

6.5 Future Research Directions

1. **Cleaner tuning protocol.** Reserve a separate early-stopping period inside each training window and add a time-aware hyperparameter search.
2. **Direct and hybrid multi-step strategies.** Train separate models for different horizon ranges and compare them with the recursive strategy to measure error accumulation [23].
3. **Probabilistic forecasts.** Use quantile objectives in LightGBM to produce prediction intervals, which support safety-stock decisions better than single-value forecasts.
4. **External drivers.** Add weather, regional school calendars and local events.
5. **Store clustering and hierarchy.** Compare the global model with models for clusters of similar stores, and reconcile forecasts across store, region and chain levels.
6. **Wider transfer learning.** Extend the multi-retailer experiment to more retailers and to daily grain where data allows, and test pre-training on large multi-retailer data followed by fine-tuning.
7. **Production deployment.** Schedule retraining on new data, monitor forecast accuracy over time, and host the application for planners.

References

[1] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021.

[2] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Hoboken, NJ, USA: Wiley, 2015.

[3] R. B. Cleveland, W. S. Cleveland, J. E. McRae, and I. Terpenning, "STL: A seasonal-trend decomposition procedure based on loess," *Journal of Official Statistics*, vol. 6, no. 1, pp. 3–73, 1990.

[4] S. J. Taylor and B. Letham, "Forecasting at scale," *The American Statistician*, vol. 72, no. 1, pp. 37–45, 2018.

[5] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 3146–3154.

[6] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[7] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[8] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "M5 accuracy competition: Results, findings, and conclusions," *International Journal of Forecasting*, vol. 38, no. 4, pp. 1346–1364, 2022.

[9] C. S. Bojer and J. P. Meldgaard, "Kaggle forecasting competitions: An overlooked learning opportunity," *International Journal of Forecasting*, vol. 37, no. 2, pp. 587–603, 2021.

[10] C. Guo and F. Berkhahn, "Entity embeddings of categorical variables," arXiv:1604.06737, 2016.

[11] D. Salinas, V. Flunkert, J. Gasthaus, and T. Januschowski, "DeepAR: Probabilistic forecasting with autoregressive recurrent networks," *International Journal of Forecasting*, vol. 36, no. 3, pp. 1181–1191, 2020.

[12] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," *International Journal of Forecasting*, vol. 22, no. 4, pp. 679–688, 2006.

[13] L. J. Tashman, "Out-of-sample tests of forecasting accuracy: An analysis and review," *International Journal of Forecasting*, vol. 16, no. 4, pp. 437–450, 2000.

[14] C. Bergmeir and J. M. Benítez, "On the use of cross-validation for time series predictor evaluation," *Information Sciences*, vol. 191, pp. 192–213, 2012.

[15] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 4765–4774.

[16] R. Fildes, S. Ma, and S. Kolassa, “Retail forecasting: Research and practice,” *International Journal of Forecasting*, vol. 38, no. 4, pp. 1283–1318, 2022.

[17] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[18] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, “Leakage in data mining: Formulation, detection, and avoidance,” *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, Art. 15, 2012.

[19] Rossmann and Kaggle, “Rossmann Store Sales,” Kaggle competition dataset, 2015. [Online]. Available: https://www.kaggle.com/c/rossmann-store-sales

[20] “Walmart Sales,” Kaggle dataset (45 stores, weekly sales, 2010–2012). [Online]. Available: https://www.kaggle.com

[21] S. M. Lundberg *et al.*, “From local explanations to global understanding with explainable AI for trees,” *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020.

[22] P. Montero-Manso and R. J. Hyndman, “Principles and algorithms for forecasting groups of time series: Locality and globality,” *International Journal of Forecasting*, vol. 37, no. 4, pp. 1632–1653, 2021.

[23] S. Ben Taieb, G. Bontempi, A. F. Atiya, and A. Sorjamaa, “A review and comparison of strategies for multi-step ahead time series forecasting based on the NN5 forecasting competition,” *Expert Systems with Applications*, vol. 39, no. 8, pp. 7067–7083, 2012.

[24] A. E. Hoerl and R. W. Kennard, “Ridge regression: Biased estimation for nonorthogonal problems,” *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.

Appendices

Appendix A: Leakage-safe feature construction

The core of the feature code, from `src/features.py`. The target is shifted **within each store** before any window is computed, so a feature only ever sees strictly earlier sales.

```
def add_lag_features(df, target_col="Sales", group_col="Store",
                     lags=(1, 7, 14, 28)):
    grouped_target = df.groupby(group_col)[target_col]
    for lag in lags:
        df[f"lag_{lag}"] = grouped_target.shift(lag)
    return df

def add_rolling_features(df, target_col="Sales", group_col="Store",
                         windows=(7, 14, 28)):
    shifted = df.groupby(group_col)[target_col].shift(1)
    for window in windows:
        roll = shifted.groupby(df[group_col]).rolling(
            window=window, min_periods=window)
        df[f"roll_mean_{window}"] = roll.mean().reset_index(level=0,
drop=True)
        df[f"roll_std_{window}"]  = roll.std().reset_index(level=0,
drop=True)
    return df
```

Appendix B: Walk-forward validation splits

Simplified from `src/validation.py`. Validation blocks are built backwards from the last date; every training window contains only earlier dates.

```
def walk_forward_splits(dates, n_folds=5, val_days=42):
    dates = pd.to_datetime(pd.Series(dates).reset_index(drop=True))
    val_blocks, val_end = [], dates.max()
    for _ in range(n_folds):
        val_start = val_end - pd.Timedelta(days=val_days - 1)
        val_blocks.append((val_start, val_end))
        val_end = val_start - pd.Timedelta(days=1)
    val_blocks.reverse()
    splits = []
    for fold, (val_start, val_end) in enumerate(val_blocks, start=1):
        train_mask = (dates < val_start).to_numpy()
        val_mask = ((dates >= val_start) & (dates <= val_end)).to_numpy()
```

```
        splits.append({"fold": fold, "train_mask": train_mask,
                       "val_mask": val_mask})
    return splits
```

Appendix C: LightGBM configuration

```
LGBMRegressor(
    objective="regression", n_estimators=1000, learning_rate=0.05,
    num_leaves=63, subsample=0.8, colsample_bytree=0.8, random_state=42,
)
# fitted on log1p(Sales); early stopping after 50 rounds without
improvement;
# predictions transformed back with expm1
```

Appendix D: Full validation results by fold

Table 13: Complete walk-forward validation results

Fold	Model	RMSE	MAE	WMAPE	MASE
1	Seasonal-naive	2,487	1,775	25.82%	0.757
1	Moving average	2,118	1,518	22.08%	0.647
1	Ridge	1,273	860	12.50%	0.366
1	LightGBM	844	594	8.64%	0.253
2	Seasonal-naive	2,421	1,879	27.97%	0.805
2	Moving average	2,035	1,435	21.36%	0.615
2	Ridge	1,047	709	10.55%	0.304
2	LightGBM	1,116	475	7.06%	0.203
3	Seasonal-naive	3,984	3,022	39.49%	1.303
3	Moving average	2,894	2,189	28.61%	0.944
3	Ridge	1,468	1,006	13.15%	0.434
3	LightGBM	920	643	8.40%	0.277
4	Seasonal-naive	3,331	2,278	31.28%	0.974
4	Moving average	2,518	1,856	25.50%	0.794
4	Ridge	1,360	903	12.41%	0.386
4	LightGBM	760	548	7.53%	0.234
5	Seasonal-naive	2,830	2,192	31.41%	0.940
5	Moving average	2,323	1,597	22.89%	0.685
5	Ridge	1,169	748	10.72%	0.321
5	LightGBM	791	540	7.73%	0.232

Fold 2 is the only fold where LightGBM's RMSE (1,116) is higher than Ridge's (1,047), even though its MAE and WMAPE are much lower. RMSE is dominated by a few very large errors,

which suggests that a small number of unusual store-days in that period were badly predicted, while the typical error remained much smaller.

Appendix E: Additional SHAP dependence plots

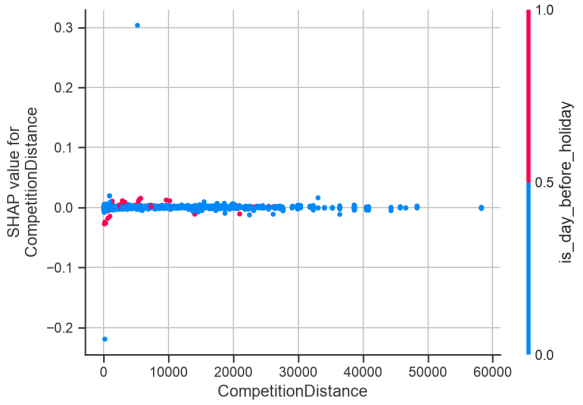


Figure 15: SHAP dependence: CompetitionDistance

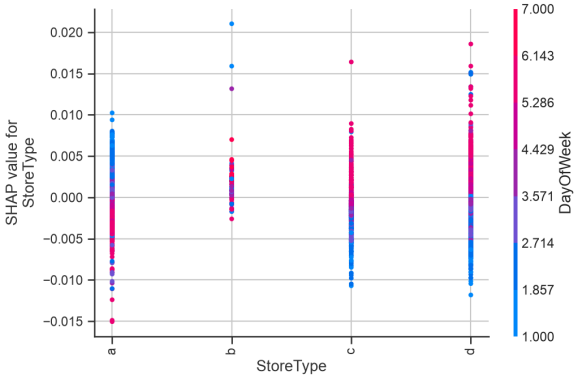


Figure 16: SHAP dependence: StoreType

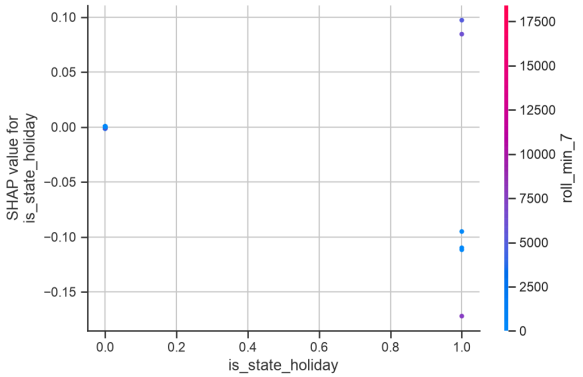


Figure 17: SHAP dependence: state holiday

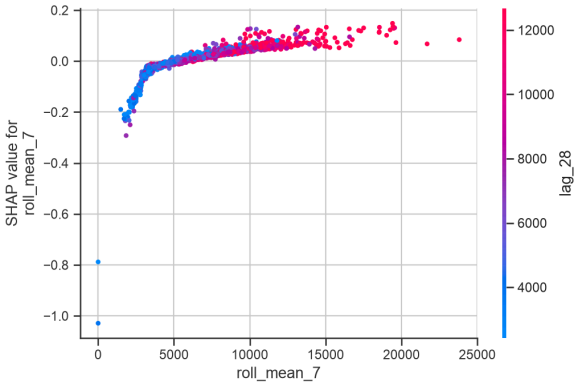


Figure 18: SHAP dependence: 7-day rolling mean

Code and Dataset Availability

Source code. The complete source code, including all notebooks, the `src/` library, the web application, unit tests and result tables, is publicly available at:
https://github.com/PrashamJ17/Store-Level-Retail-Demand-Forecasting

Datasets. The datasets cannot be redistributed with the code under Kaggle's terms, but are freely available after accepting those terms with a Kaggle account:

- Rossmann Store Sales: https://www.kaggle.com/c/rossmann-store-sales (files `train.csv`, `store.csv`, `test.csv`);
- Walmart Sales: the single-file "Walmart Sales" Kaggle dataset (`Walmart_Sales.csv`), used only for notebook 09.

Reproducing the results.

```
pip install -r requirements.txt
# place train.csv, store.csv, test.csv (and Walmart_Sales.csv) in data/raw/
pytest tests/                 # 22 tests, no data required
# run notebooks/00 ... 08 in order (09 optional)
python webapp/app.py          # then open http://127.0.0.1:8000
```

Every number in this report was produced by running this code on the real datasets.